



DEBRE BERHAN UNIVERSITY

College of Computing

Chapter 6: ASP .Net Web Application

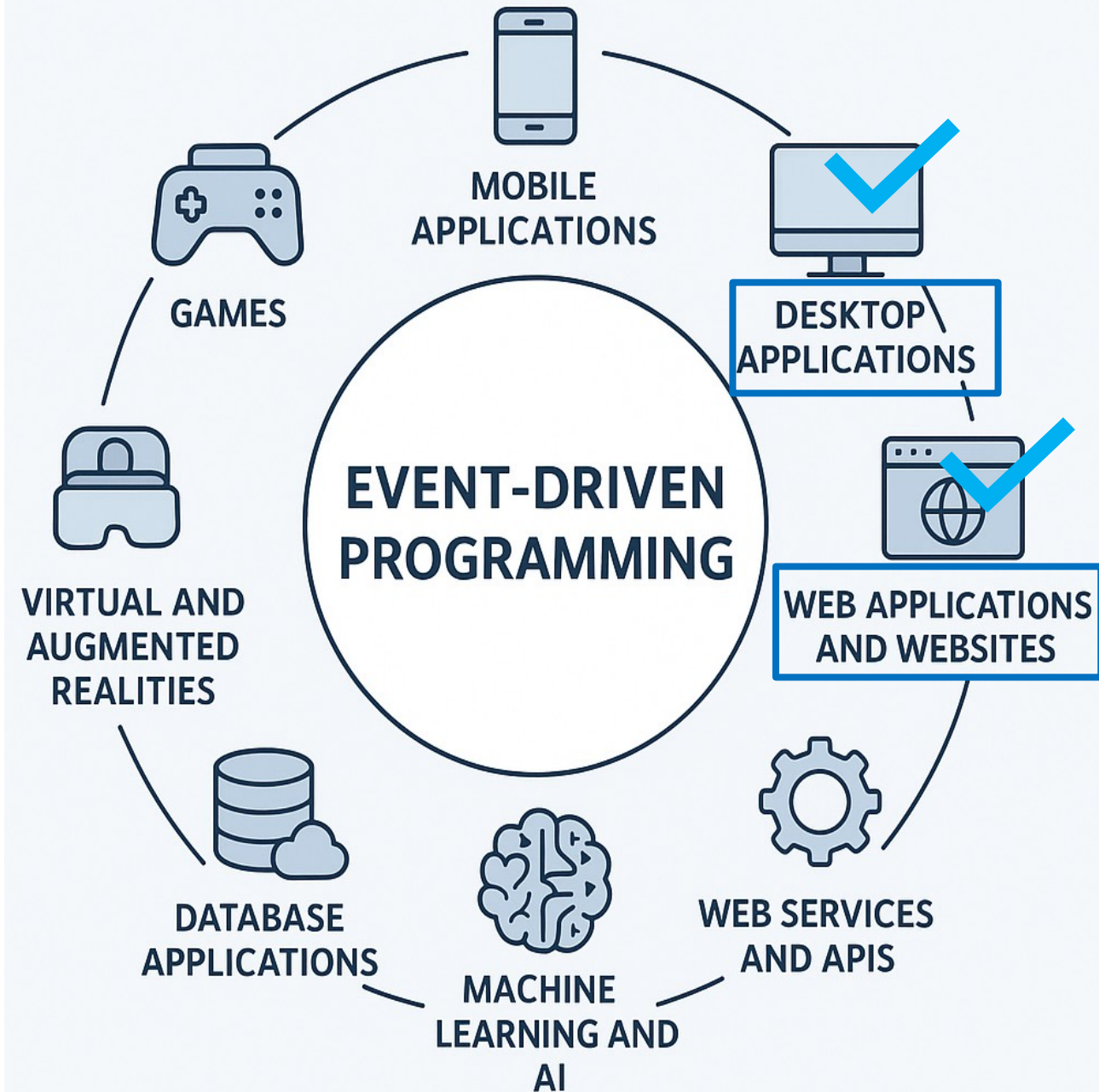


By Binyam T.

August 13, 2026

Outline

- Web Application Programming
- JWT
- MVC



Why Web Applications?

- Accessible from anywhere (browser-based)
- No installation required
- Centralized updates
- Integrates easily with APIs and databases
- Suitable for enterprise and public systems
- **Examples**
 - Inventory systems
 - Student portals
 - E-commerce websites

ASP.NET Core Overview

- **What is ASP.NET Core?**
- Cross-platform web framework by Microsoft
- Runs on Windows, Linux, macOS
- High performance and scalable
- Supports:
 - MVC
 - Web API
 - Razor Pages
 - Blazor

Types of ASP.NET Applications

- **ASP.NET Core Application Types**
- **Web API**
 - Backend only
 - Returns JSON
- **MVC Web Application**
 - Controllers + Views
 - Returns HTML pages
- **Razor Pages**
 - Page-focused model
- **Blazor**
 - C# for client-side UI
-  **This chapter focuses on MVC Web Application**

What is MVC?

- **MVC Architectural Pattern**
- **MVC = Model – View – Controller**
- **Model**
 - Data and business rules
- **View**
 - User Interface (HTML)
- **Controller**
 - Handles requests
 - Coordinates Model and View

MVC Flow Diagram (Conceptual)

■ How MVC Works

- User sends request (browser)
- Controller receives request
- Controller calls API / Model
- Data returned to Controller
- Controller passes data to View
- View renders HTML to browser

MVC in Our Course Architecture

- **Our 3-Tier Architecture**
- **Data Layer**
 - Entity Framework Core
 - Database access
- **API Layer**
 - Business logic
 - Authentication
 - REST endpoints
- **Web MVC Layer**
 - UI only
 - Calls API using HttpClient
- Clean separation of concerns

ASP.NET MVC Project Structure

- **Typical MVC Project Structure**

- **Controllers/**

- HomeController
- ProductsController

- **Views/**

- Home/
- Products/
- Shared/

- **Models/**

- ViewModels
- DTOs

- **wwwroot/**

- CSS, JS, images

- **Program.cs**

- **appsettings.json**

Controllers

- **Controller Responsibilities**

- Receives HTTP requests
- Calls API or services
- Selects which View to return
- Passes data to Views

- **Example:**

```
public IActionResult Index()  
{  
    return View();  
}
```

Views (Razor)

- **What is a View?**
- HTML + C# (Razor syntax)
- Generates dynamic web pages
- Located in **Views/ControllerName/**

```
<h2>Products</h2>
@foreach (var p in Model)
{
    <p>@p.Name</p>
}
```

Razor Syntax Basics

Razor Syntax

- @Model.Property
- @if, @foreach
- Strongly typed views

Example:

```
@model ProductDto  
<p>Price: @Model.Price</p>
```

Models in MVC

- **Model Types in MVC**
- **ViewModels**
 - Used only by Views
- **DTOs**
 - Data from API
- **Entities**
 - ❌ Not used directly in MVC
- 📌 MVC **should not** talk to DbContext

Routing in MVC

- **URL Routing**

- Default route:

`{controller=Home}/{action=Index}/{id?}`

- Examples:

- /Home/Index
- /Products/Edit/5
- /Account/Login

Forms and Data Binding

- **Handling Forms in MVC**

- Forms submit data to Controller
- Model Binding maps form fields to objects

- Example:

```
[HttpPost]
```

```
public IActionResult Create(ProductDto dto)
```

```
{
```

```
    // process data
```

```
}
```

Validation

- **Validation in MVC**

- DataAnnotations

- Server-side validation

- Client-side validation (optional)

```
[Required]  
[Display(Name = "Username")]  
public string UserName { get; set; } = "";
```

```
if (!ModelState.IsValid)  
    return View(model);
```

```
<span asp-validation-for="UserName" class="text-  
danger"></span>
```


Authentication in Web MVC

- **Authentication Concept**
- Identifies **who** the user is
- Uses:
 - Cookies (Web UI)
 - JWT (API)
- MVC uses **Cookie Authentication**
- API uses **JWT Authentication**

Login Flow (Professional)

Login Flow in Our System

1. User submits login form
2. MVC calls API /auth/login
3. API returns JWT token
4. MVC stores token in cookie claims
5. User is authenticated

Authorization

- **Authorization**

- Controls **what** a user can access
- [Authorize] attribute
- Example:

```
[Authorize]  
public IActionResult Index()  
{  
    return View();  
}
```

- Unauthenticated users → redirected to Login

Navigation Bar (UX)

- **Professional Navigation Behavior**
- **If not logged in**
 - Show Login / Register
- **If logged in**
 - Show username
 - Show Logout button
- Improves usability and security

Consuming API in MVC

Calling API from MVC

- Use HttpClient
- Prefer IHttpClientFactory
- Send JWT automatically

Example:

```
await _http.GetFromJsonAsync<List<ProductDto>>("api/products");
```

Why MVC Should Use API (Not DB)

- **Why MVC Should NOT Access Database Directly**
- Avoid duplicated business logic
- API can be reused by:
 - Mobile apps
 - Web apps
 - Desktop apps
- Better security
- Easier maintenance

Error Handling

- **Handling Errors Gracefully**
 - Show validation messages
 - Handle API failures
 - Friendly error pages
- **Example:**
 - Invalid login → “Username or password incorrect”

When to Use ASP.NET MVC

- **Use MVC When**
 - You need server-rendered pages
 - SEO (Search Engine Optimization) is important
 - Strong control over UI flow
 - Integration with existing .NET systems

Summary

■ Key Takeaways

- ASP.NET MVC is a UI framework
- Uses Model, View, Controller pattern
- Works best with Web APIs
- Authentication + Authorization are critical
- Clean architecture improves scalability

Next Practical Session

▪ **Practical Work**

- Create MVC project
- Implement Login & Register
- Consume Products API
- Protect pages using [Authorize]

Why JWT is Needed

- **Current Situation (Before JWT)**
- API login returns only **success message**
- No identity stored after login
- Any user can call API endpoints
- Products are **not protected**
-  **Problem:** API cannot know *who* is calling it

Authentication vs Authorization

- **Important Concepts**
- **Authentication**
 - Who is the user?
- **Authorization**
 - What can the user access?
- JWT helps us with **both**

What is JWT?

- **JSON Web Token (JWT)**

- A secure text token
- Generated after login
- Sent with every request
- Proves user identity

- **Think of JWT as**

- “Digital ID card”

JWT Flow in Our System

■ Login Flow

- User logs in
- API verifies username & password
- API creates JWT token
- Token sent to Web app
- Web app sends token on every API request

Step 1: Add JWT Settings (API)

- File: appsettings.json

```
"Jwt": {  
  "Key":  
    "REPLACE_WITH_A_LONG_RANDOM_SECRET_32+_CHARS",  
  "Issuer": "Inventory.Api",  
  "Audience": "Inventory.Web",  
  "ExpiresMinutes": 60  
}
```

- Explanation**
- Key** → secret used to sign token
- Issuer** → who created the token
- Audience** → who can use the token
- ExpiresMinutes** → token lifetime

Step 2: Configure JWT in Program.cs (API)

- File: Program.cs

```
var jwt = builder.Configuration.GetSection("Jwt");  
var key = jwt["Key"] ?? throw new InvalidOperationException("Jwt:Key missing.");
```

- Reads JWT settings from appsettings.json

Add JWT Authentication (API)

```
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwt["Issuer"],
            ValidAudience = jwt["Audience"],
            IssuerSigningKey =
                new SymmetricSecurityKey(Encoding.UTF8.GetBytes(key)),
            ClockSkew = TimeSpan.FromMinutes(1)
        };
    });

builder.Services.AddAuthorization();
```

Enable Authentication Middleware (API)

```
var app = builder.Build();
```

```
app.UseAuthentication();  
app.UseAuthorization();
```

- **Order matters**

- Authentication first
- Authorization second

Protect API Endpoints

- File: ProductsController.cs

```
[Authorize]
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
}
```

- Now products API requires valid JWT token

API Side Summary

■ What We Achieved

- API now issues JWT
- API validates JWT on requests
- Unauthorized users are blocked
-  API is secure

Create ASP.NET MVC Project

Create MVC Project

- Project Type: **ASP.NET Core Web App (MVC)**
- Name: Inventory.Web
- Framework: .NET 9
-  This project is **UI only**

MVC Project Structure (First Look)

- **Important Folders**

- **Controllers/**
- **Views/**
- **Models/**
- **wwwroot/**
- **Program.cs**

MVC Role in Our Architecture

- **MVC Responsibilities**

- Show HTML pages
- Handle user input
- Call API using HttpClient
- Never talk to database directly

Configure MVC Authentication

- **MVC Uses Cookie Authentication**

- Cookies store login state
- JWT stored inside cookie claims
- Secure & server-side

VC Program.cs Authentication Setup

```
builder.Services.AddAuthentication(CookieAuthenticationDefaults.AuthenticationScheme)
    .AddCookie(options =>
    {
        options.LoginPath = "/Account/Login";
        options.AccessDeniedPath = "/Account/AccessDenied";
    });
```

```
builder.Services.AddAuthorization();
```

- Configures cookie-based authentication for the MVC application
- Tell ASP.NET how users are logged in and where to redirect them when they are not authenticated
- If not authenticated: /Account/Login) or
- Authenticated but not authorized: not allowed (/Account/AccessDenied).

MVC Login Concept

■ MVC Login Process

- User enters username & password
- MVC sends credentials to API
- API returns JWT
- MVC stores JWT in cookie
- User becomes authenticated

```

public sealed class ApiAuthHeaderHandler : DelegatingHandler
{
    private readonly IHttpContextAccessor _httpContextAccessor;

    public ApiAuthHeaderHandler(IHttpContextAccessor httpContextAccessor)
        => _httpContextAccessor = httpContextAccessor;

    protected override async Task<HttpResponseMessage>
SendAsync(HttpRequestMessage request, CancellationToken ct)
    {
        var httpContext = _httpContextAccessor.HttpContext;

        if (httpContext?.User?.Identity?.IsAuthenticated == true)
        {
            // token stored as claim (see AccountController below)
            var token = httpContext.User.FindFirst("api_token")?.Value;
            if (!string.IsNullOrEmpty(token))
                request.Headers.Authorization = new
AuthenticationHeaderValue("Bearer", token);
        }

        return await base.SendAsync(request, ct);
    }
}

```

Automatically Sending JWT to the API

- What is a DelegatingHandler?
- **DelegatingHandler**
- A special class used with HttpClient
- Intercepts HTTP requests **before** they are sent
- Allows us to modify requests (headers, tokens, etc.)
- Here, we use it to add the **Authorization header**

Why Do We Need This?

- **Problem Without This Handler**
- MVC logs in correctly
- API requires JWT token
- Every API call must include:

Authorization: Bearer <token>

- Manually adding this in every request is **error-prone**

Class Declaration

```
public sealed class ApiAuthHeaderHandler : DelegatingHandler
```

Explanation

- sealed → cannot be inherited
- DelegatingHandler → intercepts HTTP requests
- Used by HttpClient

Accessing Current User (HttpContext)

```
private readonly IHttpContextAccessor _httpContextAccessor;
```

- **Why This Is Needed**
- Allows access to:
 - Logged-in user
 - Authentication cookie
 - User claims (JWT)
- MVC API calls run **outside controllers**, so we need this service.

MVC Login Concept

```
public ApiAuthHeaderHandler(IHttpContextAccessor httpContextAccessor)  
    => _httpContextAccessor = httpContextAccessor;
```

Explanation

- Dependency Injection
- ASP.NET provides the current HttpContext
- Makes user information available inside this class

AccountController (MVC)

- **Purpose of AccountController**
 - Login
 - Register
 - Logout
- This controller manages **authentication UI**

Intercepting the Request

```
protected override async Task<HttpResponseMessage>  
SendAsync(HttpRequestMessage request, CancellationToken ct)
```

- **Explanation**
- This method runs **before** the HTTP request is sent
- Perfect place to add headers
- Runs for **every API request**

Check If User Is Logged In

```
if (HttpContext?.User?.Identity?.IsAuthenticated == true)
```

- **Explanation**
- Checks whether the user is authenticated
- Prevents adding a token for anonymous users
- Avoids unnecessary errors

Reading the JWT Token

```
var token = httpContext.User.FindFirst("api_token)?.Value;
```

- **Explanation**
- JWT was stored as a **claim** during login
- Claims live inside the authentication cookie
- We retrieve the token safely from there

Adding Authorization Header

```
request.Headers.Authorization = new AuthenticationHeaderValue("Bearer", token);
```

■ **What this Does**

- Adds this HTTP header automatically:

Authorization: Bearer <JWT_TOKEN>

- Now the API can:
- Identify the user
- Authorize access

- Sending the Request

- **Explanation**

```
return await base.SendAsync(request, ct);
```

- Sends the modified request to the API
- The API receives a **secure, authenticated request**

Login Action (MVC)

```
[HttpPost]
public async Task<ActionResult> Login(LoginVm model)
{
    var result = await _api.LoginAsync(...);

    var claims = new List<Claim>
    {
        new Claim(ClaimTypes.Name, model.UserName),
        new Claim("api_token", result.Token)
    };

    await HttpContext.SignInAsync(
        CookieAuthenticationDefaults.AuthenticationScheme,
        new ClaimsPrincipal(new ClaimsIdentity(claims)));
}
```

- JWT is stored as a **claim**

Register Action (MVC)

■ Register Flow

- User fills registration form
- MVC sends data to API
- API creates user
- Redirect to Login page

Login & Register Views (Razor)

- Razor View Basics

```
@model LoginVm
```

```
<input asp-for="UserName" />
```

```
<input asp-for="Password" type="password" />
```

- Razor = HTML + C#
- Strongly typed views

Show Username & Logout (Layout)

■ Navigation Bar Logic

- If logged in → show username + Logout
- If not logged in → show Login / Register

```
@if (User.Identity.IsAuthenticated)
{
    <span>@User.Identity.Name</span>
}
```

Product CRUD (MVC)

■ ProductController (MVC)

- Index → List products
- Create → Add product
- Edit → Update product
- Delete → Remove product
- All operations call **API**

Calling API from MVC

- HttpClient Example

```
await _http.GetFromJsonAsync<List<ProductDto>>("api/products");
```

- MVC never uses DbContext
- API handles business logic

Authorize MVC Pages

- Secure MVC Pages

```
[Authorize]  
public class ProductsController : Controller  
{  
}
```

- Unauthenticated users:

-  Redirected to Login page automatically

Complete Request Flow

- **End-to-End Flow**
- Browser → MVC Controller
- MVC → API (JWT)
- API → Database
- Database → API
- API → MVC
- MVC → View (HTML)

Common Student Mistakes

- **Avoid These Errors**
- Accessing DbContext in MVC ✖
- Forgetting [Authorize]
- Not adding UseAuthentication()
- Storing JWT in JavaScript ✖

Thank You!

?